Experimental Analysis of Binary Search Models in Trees

Michał Szyfel

Gdańsk University of Technology

December 8, 2025

Outline

The Tree Search Problem

Problem Statement

Given a tree $T = (V, E)$ with a hidden target vertex x:

- Query a vertex v with cost $c(v)$
- If $v = x$: target found
- Otherwise: receive the subtree containing x
- Goal: Minimize worst-case total query cost

The Tree Search Problem

Problem Statement

Given a tree $T = (V, E)$ with a hidden target vertex x:

- Query a vertex v with cost $c(v)$
- If $v = x$: target found
- Otherwise: receive the subtree containing x
- Goal: Minimize worst-case total query cost

Problem Variants

- **Uniform costs:** $T||V||\text{COST}_{max}$ – all queries have unit cost
- **Non-uniform costs:** $T||V, c||\text{COST}_{max}$ – queries have arbitrary costs

Three-Field Notation

Notation: $\alpha||\beta||\gamma$

Classification scheme for search problems:

- α – **Search domain:** structure being searched
  - T (trees), G (graphs), P (posets)
- β – **Target set and costs:**
  - V – any vertex can be target
  - c – non-uniform query costs (omitted if uniform)
- γ – **Optimization criterion:**
  - COST_{max} – minimize worst-case cost
  - COST_{avg} – minimize average cost

Examples:

- $T||V||\text{COST}_{max}$ – uniform costs, worst-case optimization
- $T||V, c||\text{COST}_{max}$ – non-uniform costs, worst-case optimization

Hardware and Software

Computing Environment

- **Processor:** Intel Core i7-8700K (6 cores, 12 threads, 3.7 GHz)
- **RAM:** 32 GB DDR4
- **OS:** Debian Linux
- **Language:** Python 3.8+

All experiments ran single-threaded to ensure reproducible runtime measurements.

Tree Classes and Generation Methods

Random Trees (T_R):

- Generated via Prüfer sequences
- Uniform distribution over all labeled trees

Tree Classes and Generation Methods

Random Trees (T_R):

- Generated via Prüfer sequences
- Uniform distribution over all labeled trees

Bounded Degree Trees (T_Δ):

- Maximum degree $\Delta = 3$
- Random generation with degree constraint

Bounded Diameter Trees (T_D):

- Maximum diameter $D = 6$
- Random generation with diameter constraint

Tree Classes and Generation Methods

Random Trees (T_R):

- Generated via Prüfer sequences
- Uniform distribution over all labeled trees

Bounded Degree Trees (T_Δ):

- Maximum degree $\Delta = 3$
- Random generation with degree constraint

Bounded Diameter Trees (T_D):

- Maximum diameter $D = 6$
- Random generation with diameter constraint

Instance Sizes:

- Uniform costs: $n \in \{10, 11, \ldots, 1000\}$
- Non-uniform costs: $n \in \{10, 11, \ldots, 100\}$
- 10 instances per size

Cost Function Generation

Cost Distributions Tested

1. **Uniform costs ($c \sim \mathcal{U}(0,1)$):**
   - Continuous uniform distribution on $[0,1]$
   - All costs equally likely in the interval

Cost Function Generation

Cost Distributions Tested

1. **Uniform costs ($c \sim \mathcal{U}(0, 1)$):**
   - Continuous uniform distribution on $[0, 1]$
   - All costs equally likely in the interval
2. **Binomial costs ($c \sim \mathcal{B}(n = 10, p = 0.5)$):**
   - Discrete distribution with $n = 10$ trials, $p = 0.5$
   - Costs concentrate around mean value

Cost Function Generation

Cost Distributions Tested

1. **Uniform costs ($c \sim \mathcal{U}(0,1)$):**
   - Continuous uniform distribution on $[0,1]$
   - All costs equally likely in the interval
2. **Binomial costs ($c \sim \mathcal{B}(n=10, p=0.5)$):**
   - Discrete distribution with $n = 10$ trials, $p = 0.5$
   - Costs concentrate around mean value
3. **Exponential costs ($c \sim \mathcal{E}(\lambda=1)$):**
   - Continuous distribution with rate $\lambda = 1$
   - Many small costs, few large costs

Measured Performance Metrics

For All Algorithms

- **Average Runtime:** Mean execution time over all instances of size n
- **Maximum Runtime:** Worst-case execution time for size n

Measured Performance Metrics

For All Algorithms

- **Average Runtime:** Mean execution time over all instances of size n
- **Maximum Runtime:** Worst-case execution time for size n

For Uniform Cost Algorithms

- **Average Depth:** Mean depth of decision trees produced
- **Maximum Depth:** Maximum depth observed

Measured Performance Metrics

For All Algorithms

- **Average Runtime:** Mean execution time over all instances of size n
- **Maximum Runtime:** Worst-case execution time for size n

For Uniform Cost Algorithms

- **Average Depth:** Mean depth of decision trees produced
- **Maximum Depth:** Maximum depth observed

For Non-Uniform Cost Algorithms

- **Average Approximation Ratio:** Mean of $\frac{\text{ALG}}{\text{OPT}}$
- **Maximum Approximation Ratio:** Worst ratio observed

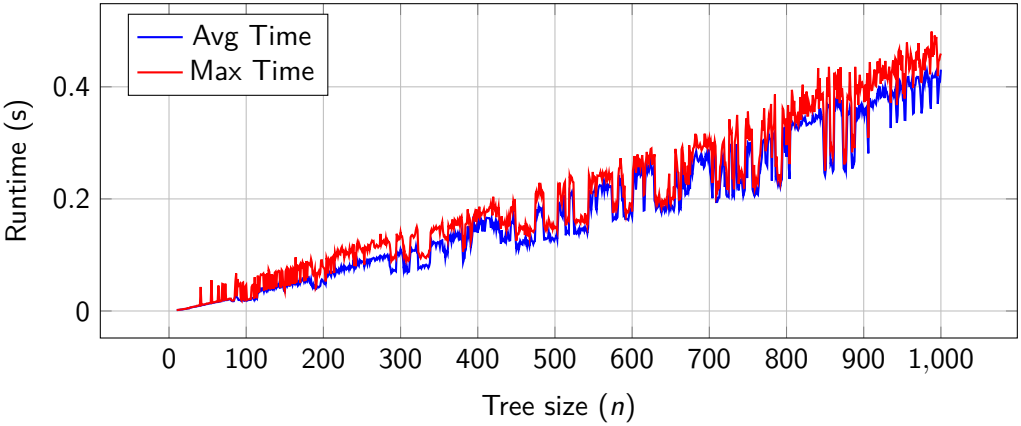
Algorithm for Uniform Costs

$T||V||\text{COST}_{max}$ – RankingBasedDT

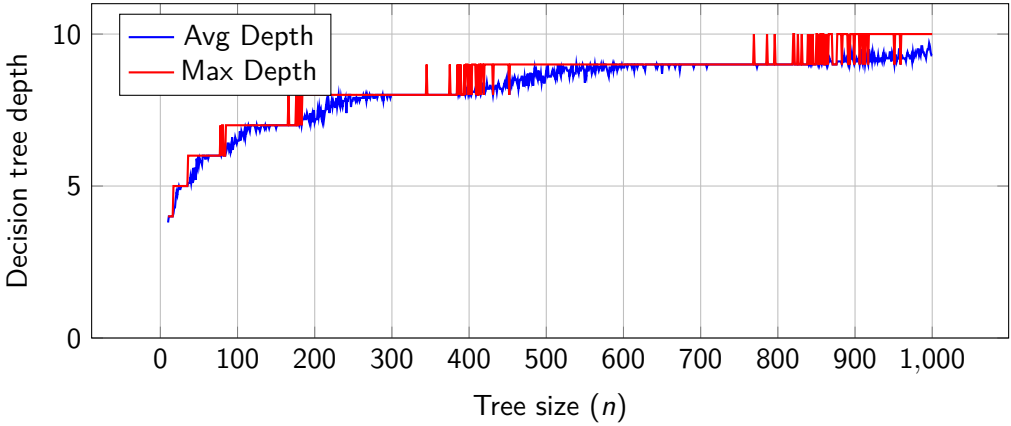
Optimal algorithm in $O(n \log n)$ time

- Based on vertex ranking coloring of trees
- Guarantees optimal decision tree depth

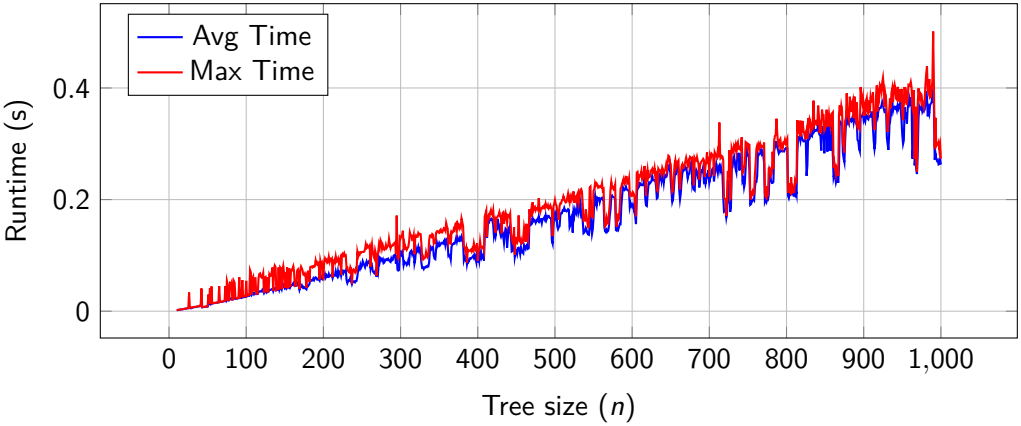
Random Trees: Runtime Performance



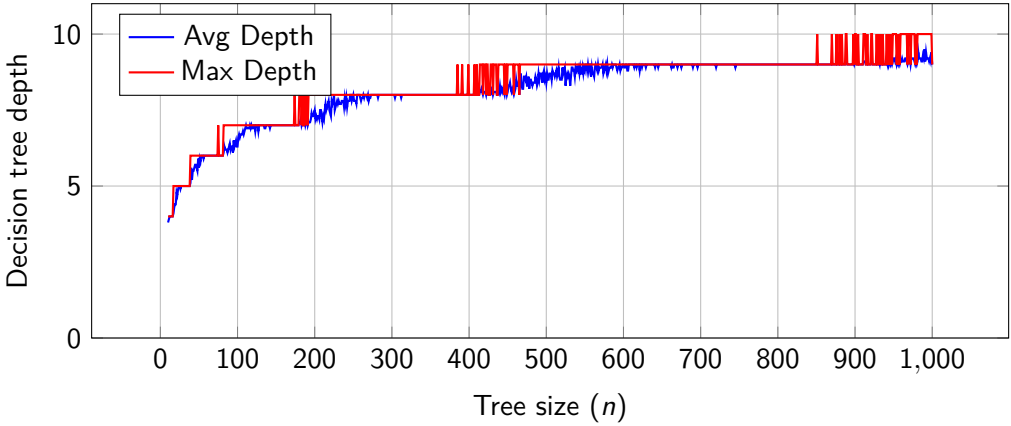
Random Trees: Decision Tree Depth



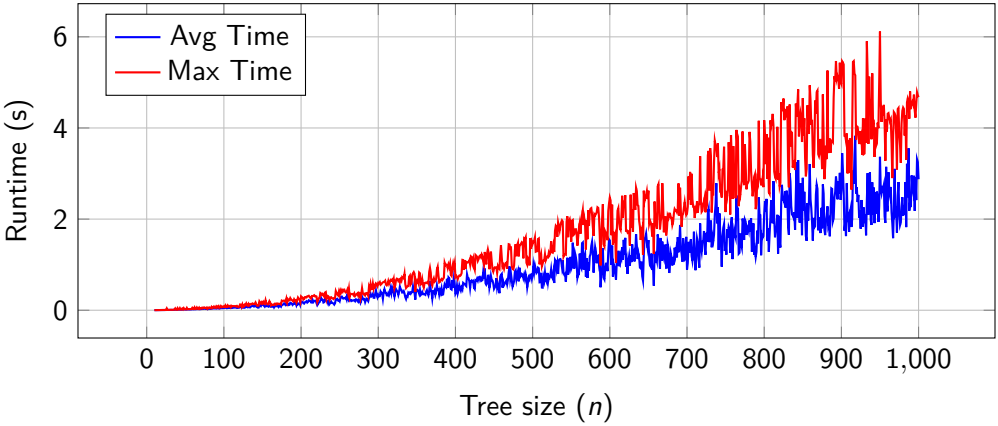
Bounded Degree Trees: Runtime Performance



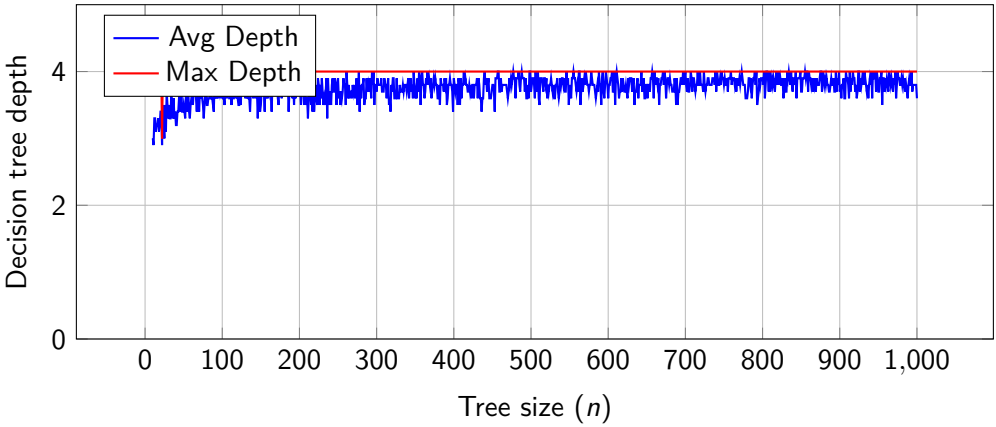
Bounded Degree Trees: Decision Tree Depth



Bounded Diameter Trees: Runtime Performance



Bounded Diameter Trees: Decision Tree Depth



RankingBasedDT Performance

Key Findings:

- Runtime: Near-linear for random and bounded degree trees
- Runtime: $O(n \log n)$ for bounded diameter
- Depth: Logarithmic for random/bounded degree
- Depth: Constant (4) for bounded diameter

RankingBasedDT Performance

Key Findings:

- Runtime: Near-linear for random and bounded degree trees
- Runtime: $O(n \log n)$ for bounded diameter
- Depth: Logarithmic for random/bounded degree
- Depth: Constant (4) for bounded diameter

Observations:

- Bounded diameter: trivial upper bound $\lceil \frac{\text{diam}(T)+1}{2} \rceil$
- Bounded degree: trivial lower bound $\log_\Delta n$
- Conjecture: For random trees: $\mathbb{E}[\text{rank}(T)] = \Theta(\log n)$ or w.h.p. $\text{rank}(T) = \Theta(\log n)$

Algorithms for Non-Uniform Costs

$T||V, c||\mathrm{COST}_{max}$

1. **Exact DP** – OPT in $O(2^n \cdot n)$ time

Algorithms for Non-Uniform Costs

$T||V, c||\mathrm{COST}_{max}$

1. **Exact DP** – OPT in $O(2^n \cdot n)$ time
2. $O(\log n / \log \log n)$-**approximation** – polynomial time

Algorithms for Non-Uniform Costs

$T||V, c||\mathrm{COST}_{max}$

1. **Exact DP** – OPT in $O(2^n \cdot n)$ time
2. $O(\log n / \log\log n)$-**approximation** – polynomial time
3. $O(\sqrt{\log n})$-**approximation** – based on QPTAS

Algorithms for Non-Uniform Costs

$T||V, c||\mathrm{COST}_{max}$

1. **Exact DP** – OPT in $O(2^n \cdot n)$ time
2. $O(\log n / \log\log n)$-**approximation** – polynomial time
3. $O(\sqrt{\log n})$-**approximation** – based on QPTAS
4. $O(\log\log n)$-**approximation** – based on QPTAS, $k^{O(\log k)} \cdot \mathrm{poly}(n)$ time

Random Trees: Exact Algorithm Runtime

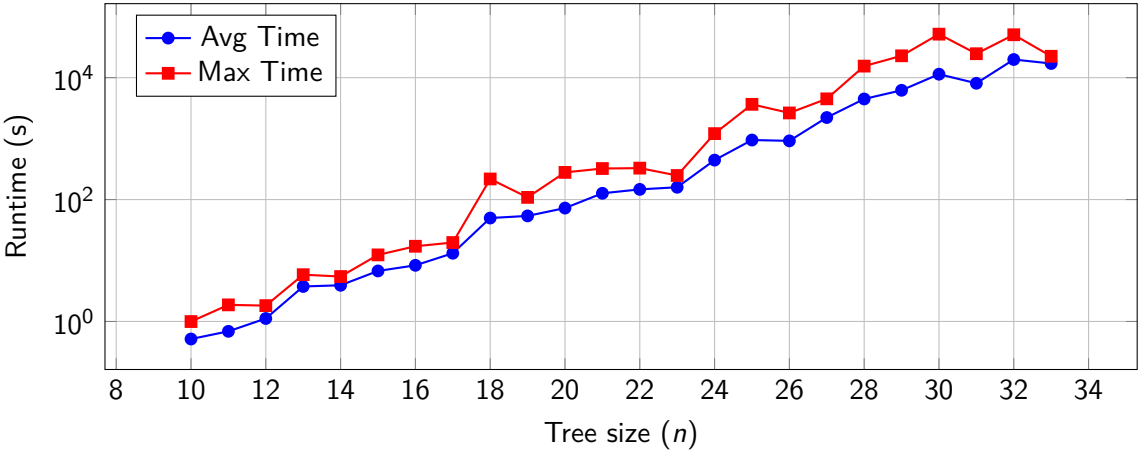


Figure: Random trees with uniform costs - exact algorithm: runtime

Random Trees: $O(\log n / \log \log n)$-Approximation Quality

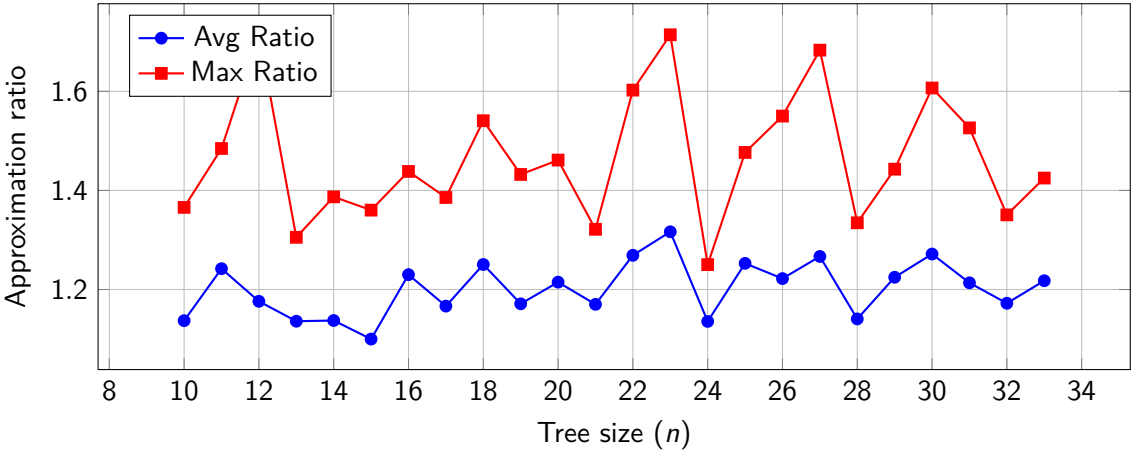


Figure: Random trees with uniform costs - $O(\log n / \log \log n)$-approximation algorithm: approximation quality

Random Trees: $O(\log n / \log \log n)$-Approximation Runtime

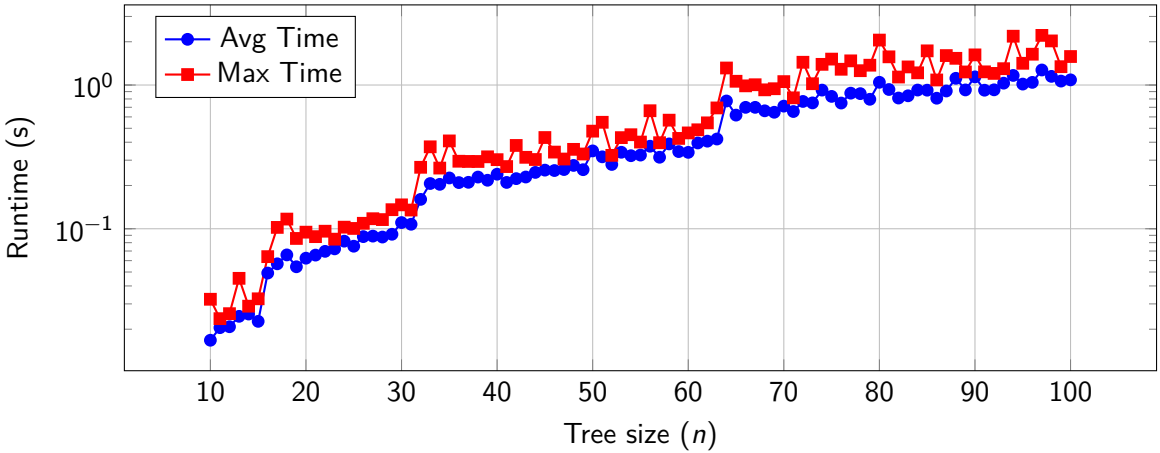


Figure: Random trees with uniform costs - $O(\log n / \log \log n)$-approximation algorithm: runtime

Random Trees: $O(\sqrt{\log n})$-Approximation Quality

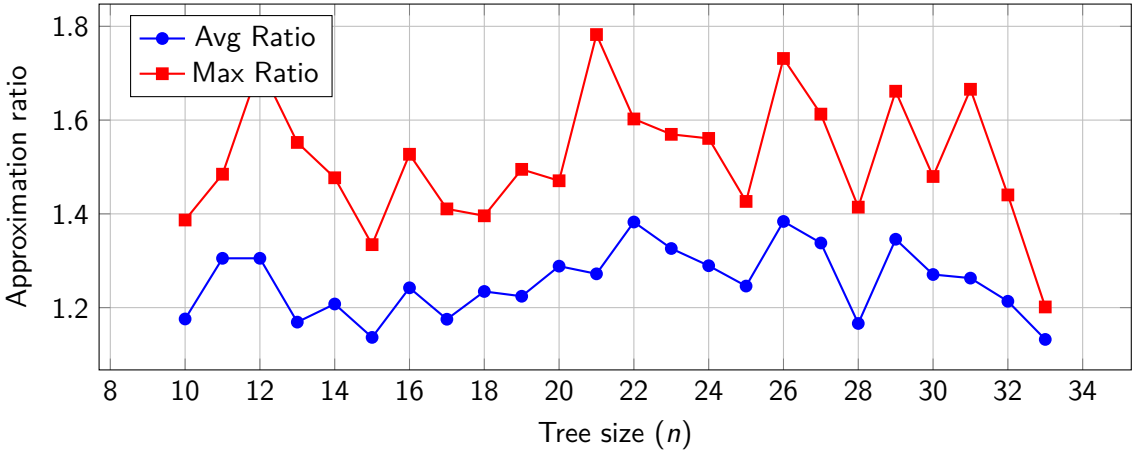


Figure: Random trees with uniform costs - $O(\sqrt{\log n})$-approximation algorithm: approximation quality

Random Trees: $O(\sqrt{\log n})$-Approximation Runtime

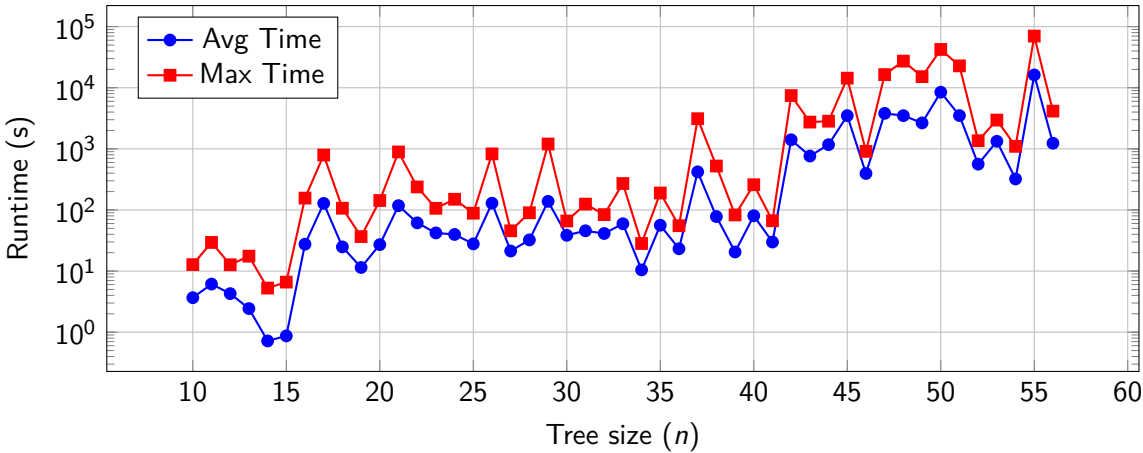


Figure: Random trees with uniform costs - $O(\sqrt{\log n})$-approximation algorithm: runtime

Random Trees: $O(\log \log n)$-Approximation Quality

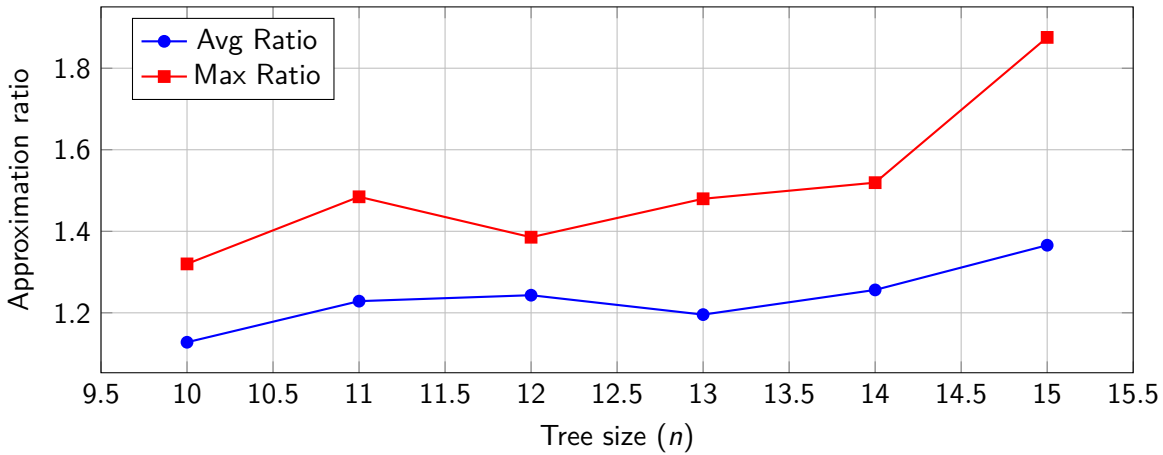


Figure: Random trees with uniform costs - $O(\log \log n)$-approximation algorithm: approximation quality

Random Trees: $O(\log \log n)$-Approximation Runtime

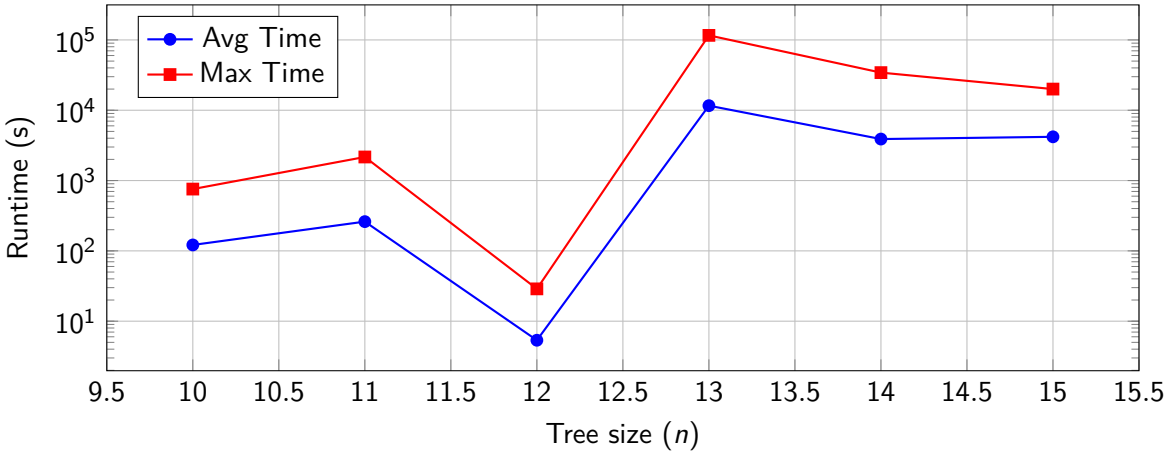


Figure: Random trees with uniform costs - $O(\log \log n)$-approximation algorithm: runtime

Exact Algorithm Runtime Analysis

Exponential function regression: $T(n) = b^n$

Fitted bases b for different tree types:

- Random trees: $b \approx 1.34 - 1.39$
- Bounded degree: $b \approx 1.35 - 1.38$
- Bounded diameter: $b \approx 1.51 - 1.61$

Exact Algorithm Runtime Analysis

Exponential function regression: $T(n) = b^n$

Fitted bases b for different tree types:

- Random trees: $b \approx 1.34 - 1.39$
- Bounded degree: $b \approx 1.35 - 1.38$
- Bounded diameter: $b \approx 1.51 - 1.61$

Practical implications:

- Much better than worst-case $O(2^n)$
- Solved instances up to $n \approx 33$ vertices
- Matches theoretical bound $O(1.42^n)$ w.h.p. for random trees

Approximation Algorithms: Quality

Approximation Ratios

All algorithms achieved:

- Average ratios: < 2.0 across all configurations
- Maximum ratios: < 3.0 in worst cases
- No significant difference between algorithms

Approximation Algorithms: Quality

Approximation Ratios

All algorithms achieved:

- Average ratios: < 2.0 across all configurations
- Maximum ratios: < 3.0 in worst cases
- No significant difference between algorithms

Surprising observation:

- $O(\log n / \log \log n)$-approximation often performed *best*
- More greedy approach works better on random instances
- $O(\log \log n)$-approximation slightly worse due to multiple cost levels

Approximation Algorithms: Runtime

Practical Usability

- $O(\log n / \log \log n)$-approximation: practical for $n \leq 100$
  - Stable runtime, no spikes
  - Recommended for practical use
- $O(\sqrt{\log n})$ and $O(\log \log n)$: limited to small instances ($n \leq 50$)
  - High variance in runtime
  - QPTAS bottleneck when rank(T) is large

Approximation Algorithms: Runtime

Practical Usability

- $O(\log n / \log \log n)$-approximation: practical for $n \leq 100$
  - Stable runtime, no spikes
  - Recommended for practical use
- $O(\sqrt{\log n})$ and $O(\log \log n)$: limited to small instances ($n \leq 50$)
  - High variance in runtime
  - QPTAS bottleneck when rank(T) is large

QPTAS behavior:

- Runtime: $n^{O(\log \text{rank}(T)/\epsilon^2)}$
- Path-like auxiliary trees cause slowdowns
- For bounded diameter: becomes PTAS with $n^{O(D/\epsilon^2)}$ time

Main Results

1. **Experimental Validation:**
   - Confirmed theoretical approximation ratios
   - Verified runtime complexities

Main Results

1. **Experimental Validation:**
   - Confirmed theoretical approximation ratios
   - Verified runtime complexities
2. **Practical Insights:**
   - Simple greedy algorithms perform well on random instances
   - Exact algorithm practical for $n \leq 33$
   - $O(\log n / \log \log n)$ recommended for larger instances

Main Results

1. **Experimental Validation:**
   - Confirmed theoretical approximation ratios
   - Verified runtime complexities
2. **Practical Insights:**
   - Simple greedy algorithms perform well on random instances
   - Exact algorithm practical for $n \leq 33$
   - $O(\log n / \log \log n)$ recommended for larger instances
3. **Structural Observations:**
   - Random trees: $\text{rank}(T) = \Theta(\log n)$ (conjectured)
   - Exact algorithm: base $b \approx 1.42$ for random trees
   - QPTAS becomes PTAS for bounded diameter trees

Open Problems and Research Directions

Algorithmic Improvements

- Design simpler QPTAS with smaller constants
- Constant-factor approximation or PTAS?
- Improve exact algorithms (similar to other combinatorial problems)

Open Problems and Research Directions

Algorithmic Improvements

- Design simpler QPTAS with smaller constants
- Constant-factor approximation or PTAS?
- Improve exact algorithms (similar to other combinatorial problems)

Theoretical Questions

- Prove $\text{rank}(T) = \Theta(\log n)$ for random trees
- Inapproximability results beyond trivial bounds
- Close gap between $O(\sqrt{\log n})$ and lower bounds

Open Problems and Research Directions

Algorithmic Improvements

- Design simpler QPTAS with smaller constants
- Constant-factor approximation or PTAS?
- Improve exact algorithms (similar to other combinatorial problems)

Theoretical Questions

- Prove $\text{rank}(T) = \Theta(\log n)$ for random trees
- Inapproximability results beyond trivial bounds
- Close gap between $O(\sqrt{\log n})$ and lower bounds

Structural Properties

- PTAS for bounded degree trees?
- Better bounds on number of connected subtrees

Conclusions

Thank you for your attention!

Questions?